

Word Order Perturbation Ablation

1. 핵심 연구 질문

AG News 분류에서 단어 순서는 실제로 얼마나 중요한가? 그리고 **LSTM**과 **Transformer Encoder**는 단어 순서가 깨졌을 때 다르게 반응하는가?

이 ablation의 목적은 단순한 성능 비교가 아니라, AG News가 문장 구조나 순서 정보를 깊게 요구하는 태스크인지, 아니면 topic-specific keyword만으로도 상당 부분 해결되는 태스크인지를 진단하는 것이다.

AG News는 **World, Sports, Business, Sci/Tech** 네 개 주제를 분류하는 뉴스 텍스트 분류 과제이다. 이 태스크에서는 **stock, company, team, oil, software, war, NASA, Microsoft** 같은 topic-specific lexical cue가 강하게 작동할 가능성이 있다. 따라서 단어 순서를 인위적으로 훼손했을 때도 성능이 크게 떨어지지 않는다면, 모델이 복잡한 sequence reasoning보다 lexical topic cue에 크게 의존했다는 해석이 가능하다.

반대로 성능이 크게 떨어진다면, AG News에서도 단순 keyword matching을 넘어서 phrase-level context, named entity relation, local syntax, class-specific expression order가 중요한 역할을 한다고 해석할 수 있다.

2. 가설

2.1 Main Hypothesis

AG News는 topic keyword 중심의 분류 태스크이므로 단어 순서를 제거해도 성능 저하는 제한적일 것이다.

2.2 Model-specific Hypothesis

LSTM은 recurrent order-processing bias가 강하기 때문에 word order perturbation에 더 민감할 수 있다. 반면 Transformer Encoder는 self-attention을 통해 전체 token set을 전역적으로 집계하므로, 같은 perturbation에서 상대적으로 덜 민감할 수 있다.

단, Transformer도 positional encoding을 사용하므로 순서를 완전히 무시하는 모델은 아니다. 특히 original order로 학습한 모델을 shuffled order에서 평가하면 Transformer 역시 distribution shift를 겪는다.

3. 실험 설계

3.1 Ablation의 기본 방향

가장 깔끔한 공식 ablation은 **train / validation / test 모두 동일한 order condition**을 적용하는 방식이다.

즉, 이 실험은 단순한 robustness test가 아니라 다음 질문을 검증한다.

단어 순서 정보가 약화되거나 제거된 조건에서도 LSTM과 Transformer Encoder가 AG News 분류를 얼마나 잘 학습할 수 있는가?

3.2 실험 조건

Condition	설명	목적
Original	원문 token order 유지	기준 성능 측정
Local Shuffle	window size 3 또는 5 안에서만 token shuffle	부분적 word order 손상
Full Shuffle	각 문서의 모든 token을 무작위 순열로 변경	word order 정보 거의 제거

가장 단순한 버전은 **Original vs Full Shuffle**만 수행하는 것이다. 다만 보고서 해석을 풍부하게 만들려면 **Local Shuffle**까지 포함하는 구성이 좋다.

4. 실험 프로토콜

4.1 공통 설정

항목	설정
Dataset	Hugging Face <code>ag_news</code>
Split	train 90%, validation 10%, official test는 최종 평가에만 사용
Tokenizer	동일 tokenizer 사용
Vocabulary	training split에서만 구축
Vocabulary size	20,000
Maximum sequence length	128
Embedding dimension	128
Batch size	64
Optimizer	Adam
Learning rate	1e-3
Epochs	8 또는 10
Loss	<code>nn.CrossEntropyLoss</code>
Seed	42, 가능하면 42 / 43 / 44의 3-seed 평균

4.2 통제해야 할 변수

Word order perturbation을 제외한 모든 조건은 고정한다.

- 동일 dataset source
- 동일 train / validation / test split
- 동일 tokenizer
- 동일 vocabulary
- 동일 maximum sequence length
- 동일 model hyperparameter
- 동일 optimizer

- 동일 learning rate
- 동일 batch size
- 동일 epoch 수 또는 early stopping rule
- 동일 evaluation metric

중요한 점은 vocabulary를 조건별로 새로 만들지 않는 것이다. Shuffle은 단어 빈도 자체를 바꾸지 않으므로, 조건마다 vocabulary를 다시 구축하면 불필요한 confound가 생긴다. 따라서 vocabulary는 **original training split**에서 **한 번만 구축**하고 모든 조건에서 공유한다.

5. 데이터 변형 방식

5.1 Original

```
# tokenization 이후 원래 순서를 그대로 사용

tokens = tokenize(text)
tokens = tokens[:max_len]
```

5.2 Local Shuffle

```
def local_shuffle(tokens, window_size=5, rng=None):
    output = []
    for i in range(0, len(tokens), window_size):
        window = tokens[i:i + window_size]
        rng.shuffle(window)
        output.extend(window)
    return output
```

Local Shuffle은 전체 문장 구조를 완전히 파괴하지 않고, local phrase order만 부분적으로 손상시킨다. 따라서 Original과 Full Shuffle 사이의 중간 조건으로 해석할 수 있다.

5.3 Full Shuffle

```
def full_shuffle(tokens, rng=None):
    tokens = tokens.copy()
    rng.shuffle(tokens)
    return tokens
```

Full Shuffle은 token identity와 token frequency는 유지하지만, token order 정보를 거의 제거한다.

5.4 Deterministic Shuffle

Epoch마다 매번 다른 shuffle을 적용하면 data augmentation 효과가 생겨 ablation 해석이 흐려질 수 있다. 따라서 sample index와 seed를 이용해 deterministic shuffle을 적용하는 것이 좋다.

```
import random

rng = random.Random(global_seed + sample_idx)
```

5.5 구현상 주의점

- `<pad>` token은 절대 shuffle하지 않는다.
- `<cls>`, `<bos>`, `<eos>` 같은 special token을 쓰는 경우 special token도 shuffle하지 않는다.
- Shuffle은 padding 이전에 적용한다.
- Truncation은 shuffle 전후 중 어느 시점에 적용할지 명확히 정해야 한다.
- 추천 방식은 `tokenize` → `truncate` → `shuffle` → `pad`이다.

6. 모델 구성

6.1 LSTM Classifier

항목	설정
Embedding	128
LSTM type	1-layer BiLSTM
Hidden size	128 per direction
Output dimension	256
Pooling	masked mean pooling으로 고정
Dropout	0.2
Classifier	Linear(256 → 4)
Loss	CrossEntropyLoss

6.2 Transformer Encoder Classifier

항목	설정
Embedding	128
Positional encoding	sinusoidal 또는 learned 중 하나로 고정
Encoder layers	2
Attention heads	4
Feedforward dimension	256

항목	설정
Pooling	masked mean pooling으로 고정
Dropout	0.2
Classifier	Linear(128 → 4)
Loss	CrossEntropyLoss

Pooling 방식은 이 ablation에서 반드시 고정해야 한다. Word order perturbation과 pooling strategy를 동시에 바꾸면, 성능 차이가 word order 때문인지 pooling 때문인지 해석하기 어렵다.

7. 평가 지표

7.1 필수 지표

- Accuracy
- Macro F1-score
- Training loss curve
- Validation loss curve
- Confusion matrix

7.2 추가 지표

Word order sensitivity를 직접 정량화하기 위해 다음 값을 추가하면 좋다.

$$\text{Delta F1} = \text{F1_perturbed} - \text{F1_original}$$

또는 상대 감소율을 사용할 수 있다.

$$\text{Order Sensitivity} = (\text{F1_original} - \text{F1_perturbed}) / \text{F1_original}$$

값이 클수록 해당 모델이 word order perturbation에 더 민감하다는 의미다.

8. 결과표 템플릿

8.1 Main Result Table

Order Condition	Model	Valid Acc	Valid Macro F1	Test Acc	Test Macro F1	Delta F1 from Original
Original	LSTM					-
Original	Transformer					-

Order Condition	Model	Valid Acc	Valid Macro F1	Test Acc	Test Macro F1	Delta F1 from Original
Local Shuffle	LSTM					
Local Shuffle	Transformer					
Full Shuffle	LSTM					
Full Shuffle	Transformer					

8.2 Class-wise F1 Table

Order Condition	Model	World F1	Sports F1	Business F1	Sci/Tech F1
Original	LSTM				
Original	Transformer				
Full Shuffle	LSTM				
Full Shuffle	Transformer				

Class-wise F1을 함께 보면 어떤 class가 word order perturbation에 더 민감한지 분석할 수 있다. 예를 들어 **Business**와 **Sci/Tech**는 용어가 겹칠 가능성이 높기 때문에 shuffle 조건에서 혼동이 커질 수 있다.

9. 예상 결과와 해석 시나리오

Case 1: Shuffle 후에도 성능이 거의 떨어지지 않음

해석:

AG News 분류는 복잡한 compositional syntax보다 topic keyword에 크게 의존한다. 두 모델 모두 sequence modeling 능력보다는 discriminative lexical cue를 활용해 분류한 것으로 보인다.

보고서 문장 예시:

The small performance degradation under full word shuffling suggests that AG News classification is largely solvable by lexical topic cues rather than fine-grained word-order reasoning.

Case 2: LSTM이 Transformer보다 더 많이 떨어짐

해석:

LSTM은 순차 hidden state 갱신 구조를 사용하므로 token order disruption에 더 민감하다. 반면 Transformer는 self-attention 기반 global aggregation이 가능해 shuffled token set에서도 상대적으로 강건할 수 있다.

Case 3: Transformer가 LSTM보다 더 많이 떨어짐

해석:

Transformer Encoder가 positional encoding과 self-attention pattern에 의존해 원래 문장 구조를 학습했을 가능성이 있다. 특히 positional encoding이 random order와 결합되면 token-position association이 불안정해질 수 있다.

Case 4: 두 모델 모두 크게 떨어짐

해석:

AG News에서도 단순 topic keyword뿐 아니라 phrase-level context, named entity relation, domain-specific expression이 중요하다. 특히 Business와 Sci/Tech, World와 Business 사이의 class overlap에서 word order가 도움이 되었을 가능성이 있다.

10. Failure Analysis 설계

Word order perturbation ablation의 failure analysis는 다음 유형으로 구성하면 좋다.

유형	설명
Type A	Original에서는 맞았지만 Full Shuffle에서는 틀린 사례
Type B	Full Shuffle에서도 맞은 사례
Type C	LSTM만 shuffle 조건에서 틀린 사례
Type D	Transformer만 shuffle 조건에서 틀린 사례
Type E	두 모델 모두 shuffle 조건에서 같은 label로 틀린 사례

각 사례에 대해 다음 항목을 기록한다.

항목	설명
Input text	원문 뉴스 텍스트
True label	정답 class
LSTM prediction	LSTM 예측
Transformer prediction	Transformer 예측
Confidence score	softmax confidence
Error reason	가능한 오류 원인
Failure pattern	두 모델이 비슷하게 실패했는지, 다르게 실패했는지

분석 질문:

1. 핵심 class cue가 하나의 단어였는가?
2. 단어 순서가 바뀌면서 의미가 모호해졌는가?
3. Business와 Sci/Tech처럼 class overlap이 있었는가?
4. 문서 앞부분 또는 뒷부분에 중요한 단서가 있었는가?
5. 셔플 후에도 모델이 특정 keyword에 과도하게 의존했는가?

11. 보고서에 넣을 수 있는 분석 문장 초안

11.1 성능 저하가 작을 때

Word order perturbation caused only a small degradation in both models, indicating that AG News classification is largely driven by topic-specific lexical cues. This suggests that, under this dataset, the advantage of sophisticated sequence modeling may be limited compared with robust keyword-based representation learning.

11.2 LSTM이 더 민감할 때

The LSTM showed a larger performance drop under shuffled inputs than the Transformer Encoder. This supports the hypothesis that recurrent sequence modeling relies more strongly on the original token order, whereas self-attention with global aggregation can remain relatively robust when the input behaves more like an unordered set of topic words.

11.3 Transformer가 더 민감할 때

Contrary to our initial expectation, the Transformer Encoder was more sensitive to word order perturbation. One possible explanation is that positional encoding and self-attention jointly learned position-dependent patterns in the original data, and randomizing token order disrupted these learned associations.

11.4 두 모델 모두 크게 떨어질 때

The large degradation in both models suggests that AG News classification is not purely a bag-of-words problem. Although topic keywords are important, local word order and phrase-level composition still provide discriminative information, especially for semantically overlapping categories such as Business and Sci/Tech.

12. 장점과 리스크

12.1 장점

- LSTM과 Transformer의 구조적 차이를 직접적으로 다룬다.
- 결과가 어떻게 나오든 해석 가능하다.
- failure analysis와 연결이 강하다.
- 성능 자체보다 실험 설계와 해석을 중시하는 과제 방향에 잘 맞는다.

12.2 리스크

- Full Shuffle은 실제 뉴스 텍스트 분포와 다소 거리가 있는 artificial perturbation이다.
- Original train → Shuffled test 방식으로 설계하면 distribution shift가 커져 해석이 조심스러워진다.
- Train / validation / test 모두 shuffled로 설계하면 robustness가 아니라 orderless condition에서의 학습 능력을 보는 실험이 된다.

따라서 보고서에서는 다음 문장을 명시하는 것이 좋다.

This ablation does not claim that real-world news text appears in shuffled order. Instead, it is a controlled diagnostic experiment designed to estimate how much the models rely on word order versus lexical topic cues.

13. 최종 추천 실험 구성

최종적으로는 다음 구성을 추천한다.

실험	구성
Main comparison	Original order에서 LSTM vs Transformer Encoder 비교
Ablation condition 1	Original order
Ablation condition 2	Local Shuffle, window size 5
Ablation condition 3	Full Shuffle
Metric	Accuracy, Macro F1, loss curve, confusion matrix
Main analysis	Delta F1, order sensitivity, class-wise confusion
Failure analysis	Original에서는 맞고 shuffle에서는 틀린 사례 중심 분석

보고서 제목 예시:

Does Word Order Matter for AG News Classification? A Controlled Comparison of LSTM and Transformer Encoder Models

이 ablation의 핵심 기여는 다음과 같이 요약할 수 있다.

This experiment tests whether the performance difference between LSTM and Transformer Encoder classifiers arises from their sequence modeling capability or from their ability to aggregate topic-specific lexical cues.

14. References

- Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation.
- Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS.
- Joulin, A., Grave, E., Bojanowski, P., & Mikolov, T. (2017). *Bag of Tricks for Efficient Text Classification*. EACL.
- Zhang, X., Zhao, J., & LeCun, Y. (2015). *Character-level Convolutional Networks for Text Classification*. NeurIPS.